Lesson 01: What Is Physical AI?

Completed: Conceptual lesson and guided Python policy exercise. **Evidence standard:** reviewed code and conversation-based answers; no uploaded local run log for final policy.

Learning objectives

• Explain the difference between perception-only AI and a physical agent.

• Recognize observation, action, policy, and environment.

• Explain closed-loop feedback as updated decisions based on new observations.

• Implement and improve a basic one-dimensional gripper policy.

Four core concepts

1. Physical AI Physical AI links intelligent decisions to actions with physical consequences. Repeating a fixed action is not necessarily feedback: feedback requires checking what happened and using that information.

2. Observations and actions Observations are sensor information: camera detections, joint measurements, and touch readings. Actions are commands, such as moving an arm or closing a gripper. A command may not have the intended effect.

3. Policies A policy chooses an action using available observations. It may be a simple rule or a trained model. Here it chooses how far the gripper should move.

4. Environment and feedback After an action, the gripper or target may change position. Observe again, compute updated distance, choose the next action, and repeat.

Feedback loop

OBSERVE	DECIDE	ACT	REPEAT
Measure distance	Choose bounded step	Move gripper	Measure again

Guided Python exercise

The gripper begins at 0 cm. A cup remains to the right. Each step is no more than 2 cm. The original code always moved by 2 cm, causing overshoot for targets such as 9 cm. The learner corrected it by limiting movement to the remaining distance.

```
gripper_position = 0
cup_position = 15
while True:
    distance = cup_position - gripper_position  # Observe
    if distance <= 0:                          # Decide: stop
        print("Gripper reached the cup")
        break
    movement = min(2, distance)                # Decide: action
    gripper_position += movement                # Act
    print("Gripper:", gripper_position, "Cup:", cup_position)
```

Key insight: `min(2, distance)` chooses the smaller of the maximum step and remaining distance. For 1 cm remaining, the policy moves just 1 cm.

Test cases: expected trajectories (not execution logs)

Target	Expected positions (cm)	Expected result
5	0, 2, 4, 5	Reach target
9	0, 2, 4, 6, 8, 9	Reach target
10	0, 2, 4, 6, 8, 10	Reach target
15	0, 2, 4, 6, 8, 10, 12, 14, 15	Reach target

Assessment and evidence

• Concept quiz: observation/action classification 3 of 3 correct.

• The learner identified the policy action toward the cup and the need for fresh observations when a target moves.

• The initial code was reviewed; the bounded-step correction was authored after guided explanation.

• Local execution of the final Python file was not supplied; this PDF does not claim that it was run.

• Isaac Sim and Isaac Lab setup verification occurred separately and does not constitute simulation deployment of the lesson policy.

Assumptions and limitations

This toy program uses one dimension, a stationary cup, a rightward target, perfect position measurement, and perfect motion. It omits collision handling, grasping, dynamics, and actuator delay. These will be introduced progressively.

Active recall: review without looking up answers

1. How is feedback different from repeated action?

2. Which are observations: camera data, a motor command, force sensor data?

3. What does a robot policy choose?

4. Why would fixed +2 cm steps fail for a cup at 9 cm?

5. If the gripper is at 6 cm and the cup moves to 15 cm, what is the new distance and the next step?

Review and next steps

Review after approximately 1 day, 1 week, and 3 weeks. Run `robot_policy.py` locally, save a terminal transcript, and attach it to the evidence record. Next: Lesson 02 - representing an environment with 2D and 3D coordinates.